UNIVERSITY OF SRI JAYAWARDENAPURA

Faculty of Technology

Department of Information and Communication Technology

WEB-NATIVE 3D VIRTUAL OFFICE ASSISTANT

A Thesis Submitted in Partial Fulfilment of the Requirements for the Degree of Bachelor of Information and Communication Technology (Hons)

Group 08

Dewmini L.G.N.	ICT/21/826
Walimuni W.D.H.D.	ICT/21/938
Thilakarathna G.S.D.P.	ICT/21/931

2026

Declaration

We declare that this thesis is our own work and has not been submitted in any form for another degree or diploma at any university or institution. Information derived from the published and unpublished work of others has been acknowledged in the text and in the list of references.

Dewmini L.G.N.	Walimuni W.D.H.D.	Thilakarathna G.S.D.P.
……..	……..	……..
Date: ……	Date: ……	Date: ……

Acknowledgement

We wish to express our sincere gratitude to our academic supervisors and the Department of Information and Communication Technology, Faculty of Technology, University of Sri Jayawardenapura, for their guidance and support throughout this research project.

We are also grateful to the developers of the open-source libraries that form the technological foundation of our system, including Three.js, React Three Fiber, YUKA, and the Groq SDK.

Abstract

Current virtual office and collaboration platforms lack the integration of spatial intelligence with Large Language Model (LLM)-powered virtual agents, resulting in environments that fail to leverage the natural spatial reasoning humans use in physical offices. This thesis presents the design, implementation, and evaluation of a **web-native 3D virtual office assistant** that fundamentally integrates spatial intelligence capabilities with LLM-powered agents to create intelligent office automation within immersive 3D environments.

The system is built entirely on web-native technologies—Next.js 16, React 19, Three.js, React Three Fiber, and the YUKA game-AI library—ensuring cross-device accessibility without specialised installations. AI agents are driven by the Groq-hosted Llama 3.1-8B-Instant model and operate through a custom `ClientBrain` pipeline that combines rate-limited LLM inference with a client-side IndexedDB-backed memory stream featuring heuristic retrieval and automatic reflection/compaction. Spatial navigation employs an A* pathfinding algorithm over a pre-configured waypoint graph mapped to the virtual office layout, while a task queue state machine coordinates multi-phase interactions such as object pickup, transport, and placement on designated surfaces.

The virtual office environment comprises procedurally constructed rooms—a lobby, open workspaces, storage room, and conference room—populated with interactive objects (files, laptops, printers, coffee stations) managed through a centralised `InteractableRegistry`. Human users navigate the environment via a third-person controller with physics-based collision detection, while AI agents autonomously patrol, follow users, and organise office resources based on LLM-generated decisions that include spatial coordinates and target identifiers.

Keywords: Spatial Intelligence, Large Language Models, Virtual Office, 3D Web Application, Embodied AI, Human-AI Collaboration, React Three Fiber, YUKA AI, A* Pathfinding

Table of Contents

Declaration 2

Acknowledgement 3

Abstract 4

List of Figures 8

List of Tables 9

List of Abbreviations 10

1 Introduction and Background 1

1.1 Introduction 1

1.2 Motivation for the Study 2

1.3 Research Problem 2

1.4 Scope of the Study 3

1.5 Objectives of the Study 3

1.5.1 Main Objective 3

1.5.2 Specific Objectives 3

1.6 Significance of the Research 4

1.6.1 Conceptual and Theoretical Significance 4

1.6.2 Practical Significance 4

1.7 Expected Outcomes 4

2 Literature Review 5

2.1 Evolution of Virtual Office Environments and 3D Workspaces 5

2.2 Large Language Models (LLMs) in Virtual Agents 5

2.3 Spatial Intelligence and 3D Navigation for Embodied AI 5

2.4 Human-AI Collaboration in Virtual Environments 5

2.5 Gap Identification 6

3 Methodology 6

3.1 Research Design and Approach 6

3.2 Technology Stack and Frameworks 6
3.2.1 Programming Languages and Core Frameworks 6
3.2.2 3D Rendering and Spatial Computing 6
3.2.3 AI and Navigation Components 7
3.2.4 Backend and Data Management 7
3.2.5 Development and Testing Tools 7
3.3 Data Collection Methods 8
3.3.1 Performance Metrics Collection 8
3.3.2 Telemetry System Implementation 8
4 System Design and Architecture 8
4.1 System Architecture Overview 8
4.1.1 Presentation Layer 8
4.1.2 Agent Intelligence Layer 9
4.1.3 Virtual Environment Layer 10
4.1.4 Backend Services Layer 11
4.2 Key System Components 11
4.2.1 Spatial Intelligence Engine 11
4.2.2 LLM Agent Controller 11
4.2.3 3D Navigation System 11
4.2.4 Human-Agent Interaction Manager 12
4.3 UML and Workflow Diagrams 12
4.3.1 Data Flow 12
5 Implementation 13
5.1 Web-Native 3D Environment Setup 13
5.2 Integrating the LLM Agent Controller 13
5.3 Implementing Spatial Reasoning and Navigation Algorithms 14
5.4 Backend and Real-Time Synchronization Setup 15

5.5 Integrating the Telemetry and Monitoring System 15
6 Results and Evaluation 16
6.1 Experimental Setup 16
6.2 Spatial Navigation Accuracy 16
6.3 LLM Decision Quality 16
6.4 Rendering Performance 16
6.5 Human-Agent Collaboration Metrics 17
7 Discussion 17
7.1 Interpretation of Results 17
7.2 Limitations 17
7.3 Comparison with Existing Systems 18
7.4 Recommendations for Future Work 18
8 Conclusion 18
8.1 Summary 18
8.2 Contributions 19
8.3 Final Remarks 19
References 20
Appendices 21
8.4 Appendix A: Project Directory Structure 21
8.5 Appendix B: Dependency Table 22
8.6 Appendix C: Environment Variables 23

List of Figures

To be populated after final figure numbering.

List of Tables

To be populated after final table numbering.

List of Abbreviations

Abbreviation	Full Form
LLM	Large Language Model
AI	Artificial Intelligence
NLP	Natural Language Processing
FSM	Finite State Machine
API	Application Programming Interface
IDB	IndexedDB
WebGL	Web Graphics Library
R3F	React Three Fiber
TPS	Third-Person Shooter (camera style)
A*	A-Star Pathfinding Algorithm
RPM	Requests Per Minute
CSV	Comma-Separated Values
OBB	Oriented Bounding Box
SDK	Software Development Kit

1 Introduction and Background

1.1 Introduction

The convergence of web technologies, spatial intelligence, and Large Language Models (LLMs) presents unprecedented opportunities for creating immersive virtual work environments. While traditional virtual assistants operate in two-dimensional interfaces, the integration of 3D spatial reasoning with LLM-powered agents opens new frontiers in human–computer interaction within virtual office spaces.

This research focuses on developing a **web-native 3D virtual office assistant** that fundamentally integrates spatial intelligence capabilities with LLM-powered virtual agents to create intelligent office automation within immersive 3D environments. The system addresses the critical gap between spatial reasoning, natural language understanding, and productivity workflows while enabling both AI agents and human users to collaborate within shared 3D virtual office spaces.

Spatial intelligence—defined as the ability to visualise, understand, and manipulate objects in space—serves as the foundational cognitive framework for this research. Our virtual agents demonstrate spatial intelligence by navigating 3D environments, understanding spatial relationships between office resources (files, documents, meeting rooms), and executing location-aware tasks. This spatial cognition enables agents to reason about *where* resources are located, *how* to navigate to them, and *what* spatial relationships exist between different office elements.

The motivation stems from the evolving nature of remote and hybrid work environments where virtual collaboration is becoming essential. Current virtual meeting platforms lack the spatial context and intelligent automation that could enhance productivity. By developing LLM agents that operate within 3D virtual offices, we create environments where AI assistants can understand and manipulate spatial information while collaborating with human workers in shared virtual spaces.

1.2 Motivation for the Study

The global shift toward remote and hybrid work models has exposed fundamental limitations in existing collaboration tools. Video conferencing platforms provide flat, two-dimensional interfaces that lack spatial context, while document management systems operate independently of any spatial metaphor. Physical offices naturally organise information spatially—files are in cabinets, notes are on whiteboards near relevant desks, and meeting rooms contain project-specific materials. This spatial organisation is entirely lost in current digital collaboration tools.

Our system addresses this gap by creating a virtual office where spatial relationships are first-class citizens. AI agents can respond to commands such as "retrieve the contract from the filing cabinet near the conference room" or "organise project documents by spatial proximity," demonstrating practical capabilities that transform the virtual office experience.

1.3 Research Problem

Current virtual office and collaboration systems lack the integration of spatial intelligence with LLM-powered virtual agents, resulting in environments that fail to leverage the natural spatial reasoning humans use in physical offices. This fundamental limitation manifests in five critical challenges:

1. **Spatial Intelligence Gap:** Existing virtual assistants operate without spatial context, unable to understand location-based relationships between office resources.
2. **LLM Integration Limitation:** Current 3D virtual environments lack intelligent agents capable of natural language interaction combined with spatial reasoning.
3. **Collaborative Virtual Office Deficiency:** No existing systems successfully combine human users and AI agents working together in shared 3D office environments.
4. **Spatial Task Execution Complexity:** Virtual environments cannot execute tasks requiring understanding of 3D spatial relationships.
5. **Natural Spatial Navigation Absence:** Current virtual office tools lack agents that can navigate 3D spaces intelligently while understanding spatial references in human language.

1.4 Scope of the Study

This research encompasses the design, implementation, and evaluation of a comprehensive 3D virtual office system with spatially intelligent LLM agents. The scope includes: development of LLM-powered virtual agents with spatial reasoning capabilities; implementation of spatial intelligence algorithms for 3D office layouts; integration of natural language processing for human–agent communication; creation of collaborative virtual office spaces; development of spatial task execution systems; implementation of 3D navigation and pathfinding; design of spatially-aware workflows; and quantitative evaluation of spatial reasoning accuracy.

The study excludes advanced photorealistic rendering, complex multi-user voice communication, external enterprise database integration, haptic feedback, real-time motion capture, and VR headset-specific optimisations.

1.5 Objectives of the Study

1.5.1 Main Objective

To design, implement, and evaluate a web-native 3D virtual office assistant system that integrates spatial intelligence capabilities with LLM-powered virtual agents, enabling intelligent spatial reasoning, natural language interaction, and collaborative task execution within immersive virtual office environments, while demonstrating measurable improvements in spatial task accuracy and human–agent collaboration effectiveness.

1.5.2 Specific Objectives

1. **Develop Spatially Intelligent LLM Agents:** Design and implement LLM-powered virtual agents capable of spatial reasoning, 3D navigation, and location-aware task execution, achieving ≥85% accuracy in spatial relationship understanding.
2. **Implement 3D Spatial Intelligence Framework:** Create algorithms enabling agents to understand spatial relationships, navigate 3D office layouts, and execute location-based tasks with ≥90% spatial accuracy.

3. **Integrate Natural Language Spatial Commands:** Develop NLP capabilities allowing humans to communicate with agents using spatial references, achieving ≥80% command interpretation accuracy.
4. **Create Collaborative Virtual Office Environment:** Implement shared 3D office spaces where human users and AI agents can work simultaneously with real-time spatial awareness.
5. **Validate Spatial Intelligence Performance:** Conduct comprehensive evaluation demonstrating statistically significant improvements ($p < 0.05$) in spatial task completion rates.
6. **Demonstrate Human–Agent Spatial Collaboration:** Evaluate collaborative scenarios measuring task efficiency and collaboration quality improvements.

1.6 Significance of the Research

1.6.1 Conceptual and Theoretical Significance

This research contributes to the theoretical understanding of how spatial intelligence can be integrated with natural language processing in embodied AI systems. It establishes a conceptual framework for LLM-powered agents operating in 3D virtual environments with spatial awareness.

1.6.2 Practical Significance

The practical significance lies in creating accessible, web-native intelligent virtual workspaces that can enhance remote collaboration. The system demonstrates that advanced spatial reasoning capabilities can be achieved in standard web browsers without specialised hardware.

1.7 Expected Outcomes

1. A functional web-native 3D virtual office environment with spatially intelligent AI agents.
2. A spatial intelligence framework for agent navigation and task execution.
3. Performance benchmarks for spatial reasoning in virtual office applications.
4. Empirical evidence of improved human–AI collaboration effectiveness.

2 Literature Review

2.1 Evolution of Virtual Office Environments and 3D Workspaces

Virtual office environments have evolved significantly from simple 2D interfaces to immersive 3D workspaces. Early systems focused on text-based collaboration, gradually incorporating graphical elements. The advent of WebGL and libraries such as Three.js enabled browser-based 3D rendering, while frameworks like React Three Fiber brought declarative, component-based paradigms to 3D scene composition.

2.2 Large Language Models (LLMs) in Virtual Agents

Park et al. (2023) demonstrated generative agents capable of simulating human behaviour through LLM-powered architectures that maintain memory, reflection, and planning capabilities. Their work established that LLMs can produce believable agent behaviours when combined with appropriate memory and retrieval mechanisms. More recently, DeepMind (2025) developed generalist AI agents for 3D virtual environments, demonstrating the potential for embodied AI in spatial contexts.

2.3 Spatial Intelligence and 3D Navigation for Embodied AI

Research in spatial reasoning for AI has made significant progress. Yang et al. (2024) explored how multimodal large language models perceive and recall spaces, while Sun et al. (2025) enhanced LLMs with spatial referring expressions for visual understanding. Liu et al. (2025) demonstrated photo-realistic 3D world creation through LLM agents, and Wu and Deng (2025) investigated the spatial representation of LLMs in 2D scenes. Duan et al. (2022) surveyed embodied AI implementations across various simulators, providing a comprehensive overview of the field.

2.4 Human-AI Collaboration in Virtual Environments

Human–AI collaboration in virtual environments remains an active research area. Current systems primarily focus on either the AI capabilities or the virtual environment aspects independently, without integrating both within a unified framework designed for office productivity.

2.5 Gap Identification

Despite advances in both embodied AI and spatial intelligence, several critical gaps remain:

Gap	Description
Gap 1	Integration of spatial intelligence with LLM agents in virtual offices remains unaddressed.
Gap 2	Web-native implementations for cross-device accessibility are limited.
Gap 3	Practical human–AI spatial collaboration in office productivity is unexplored.
Gap 4	Frameworks for complex spatial task execution combining NLP, navigation, and manipulation are absent.
Gap 5	Performance benchmarks for virtual office assistant spatial intelligence do not exist.

This research addresses all five gaps by developing a comprehensive web-native system that integrates spatial intelligence with LLM capabilities specifically for virtual office productivity.

3 Methodology

3.1 Research Design and Approach

This study employs a developmental and experimental research design combining system development with empirical evaluation. The approach integrates quantitative methods for performance measurement with controlled experimental procedures through four primary phases: (1) system architecture development, (2) implementation and integration, (3) performance optimisation, and (4) experimental validation.

3.2 Technology Stack and Frameworks

3.2.1 Programming Languages and Core Frameworks

The system is built with **TypeScript** for type-safe development, **React 19** for component-based architecture, and **Next.js 16** with server-side rendering and server actions for secure API communication. The project uses `pnpm` as the package manager.

3.2.2 3D Rendering and Spatial Computing

- **Three.js 0.181** serves as the foundational WebGL rendering engine.

- **React Three Fiber 9.4** provides declarative 3D scene composition via JSX.
- **@react-three/drei 10.7** supplies extended 3D utilities including `Html`, `Text`, `Environment`, `Stats`, and `AdaptiveEvents`.
- **WebGL 2.0** with `ACESFilmicToneMapping` for advanced rendering.

3.2.3 AI and Navigation Components

- **YUKA 0.7.8** implements autonomous agent steering behaviours, navigation graphs, and A* pathfinding.
- **Groq SDK 0.37** connects to the Groq cloud for LLM inference using the `llama-3.1-8b-instant` model.
- A custom `ClientBrain` class orchestrates the perception–reasoning–action loop with rate limiting (token-bucket, 5 RPM).
- An `AgentTaskQueue` state machine manages multi-phase task execution.

3.2.4 Backend and Data Management

- **Next.js Server Actions** (`"use server"`) handle API calls securely on the server, protecting API keys.
- **IndexedDB** (via the `idb` library v8) provides client-side persistent storage for the agent memory stream.
- **CSV-based interaction logging** records all LLM API calls with timestamps, tokens, latency, and session identifiers.
- **Zustand 5.0** provides global state management with a single `gameStore`.

3.2.5 Development and Testing Tools

- **ESLint 9** with `eslint-config-next` for code quality.
- **TypeScript 5** for static type checking.
- **Lucide React** for iconography in the UI layer.

3.3 Data Collection Methods

3.3.1 Performance Metrics Collection

The system collects: task success/failure rates as percentage completion; task completion times with millisecond precision; path efficiency as deviation from optimal routes; frame rate monitoring via the `Stats` component; memory usage tracking; and CPU/GPU frame timing.

3.3.2 Telemetry System Implementation

All LLM interactions are logged via the `agent-logger.ts` module to CSV files in the `logs/` directory. Each log entry captures 17 fields including `timestamp`, `session_id`, `request_id`, `agent_type`, `request_content`, `response_content`, `processing_time_ms`, `input_tokens`, `output_tokens`, `model_version`, and error details. This structured logging enables post-hoc analysis of agent decision quality and API performance.

4 System Design and Architecture

4.1 System Architecture Overview

The system architecture follows a four-layer design, with each layer responsible for a distinct concern. All layers communicate through well-defined TypeScript interfaces and a centralised Zustand state store (`gameStore.ts`, 391 lines).

4.1.1 Presentation Layer

The presentation layer is built with React 19 and Next.js 16. The root component (`page.tsx`) renders two parallel sub-trees:

- **3D Canvas** (`Scene.tsx`): A React Three Fiber `Canvas` element with shadow maps, adaptive DPR (1–1.5×), ACES Filmic tone mapping, and a city environment preset. It composes `OfficeHub`, `Robot` (player), two `AIRobot` instances, `YukaSystem`, `DebugCrosshair`, `ObstacleVisualizer`, `PlacingAreaMarkers`, `ObjectHighlighter`, and a `CameraRig`.
- **HTML Overlay** (`Overlay.tsx`, 542 lines): A glassmorphic UI layer containing HUD controls, a minimap canvas, an inspector panel, inventory display, notification toasts, and the `GameMenu` with settings and keybinding remapping.

The `CameraRig` component implements a third-person camera with over-the-shoulder offset (`[2.5, 0.5, 12.0]`), configurable yaw/pitch via pointer lock, ray-cast collision avoidance against scene geometry, and an inspector mode that smoothly orbits a selected AI agent.

4.1.2 Agent Intelligence Layer

This layer comprises four core classes:

- **`ClientBrain`** (`ClientBrain.ts`): Manages the Perception → Reasoning → Action cycle. Each AI agent instantiates its own `ClientBrain` with a unique session ID. It constructs an `AgentContext` (position, nearby entities, current behaviour), retrieves relevant memories from the `MemoryStream`, invokes the server-side `generateAgentThought` action, parses the JSON response into an `AgentDecision`, and stores the decision as a new memory. Rate limiting is enforced via a token-bucket `RateLimiter` (5 requests per 60 seconds).
- **`agent-core.ts`** (Server-Side): The `processAgentThought` function constructs a detailed LLM prompt with the agent's personality ("professional, efficient, warm"), spatial context (position, entity table in Markdown), memory context, and a structured JSON output schema specifying seven possible actions: `MOVE_TO`, `WAIT`, `WANDER`, `FOLLOW`, `INTERACT`, `DROP`, and `PLACE_AT`. It uses the Groq SDK with `llama-3.1-8b-instant` (temperature 1.0, max 8192 tokens) and supports automatic API key rotation on 429/401 errors.
- **`AgentTaskQueue`** (`AgentTaskQueue.ts`, 363 lines): A per-agent state machine with phases: `IDLE`, `NAVIGATING_TO_ITEM`, `PICKING_UP`, `NAVIGATING_TO_AREA`, `PLACING`, and `DONE`. Each update cycle returns a `SteeringCommand` (`FOLLOW_PATH`, `ARRIVE`, `STOP`, `NONE`) that the YUKA vehicle consumes. An `AgentTaskRegistry` singleton allows the player controller to push tasks to any agent by ID.
- **`MemoryStream`** (`MemoryStream.ts`): Implements a generative-agent-style memory system with four memory types (`OBSERVATION`, `DIALOGUE`, `THOUGHT`, `ACTION`). Heuristic importance scores (1–10) are computed without LLM calls. Retrieval uses a weighted formula: `score = importance × 0.7 + recency × 0.3`, with exponential time decay. When the memory count exceeds 400 (of a 500 cap), a reflection process summarises the oldest 50 memories into a single high-importance "Insight" via a separate LLM call and prunes the originals.

4.1.3 Virtual Environment Layer

- **`OfficeHub`** (`OfficeHub.tsx`, 1298 lines): The procedural office builder. It constructs the floor plan from the `wall_layout.md` specification: outer shell (200 × 150 units), internal partitions creating four zones (Lobby, Open Office, Storage Room, Conference Room), glass lobby partitions, and automatic sliding doors. Each zone has independent lighting controlled by wall switches.
- **`Furniture.tsx`** (1552 lines): 14 furniture components— `CeilingLight`, `WallSwitch`, `OfficeChair` (with sit interaction), `ConferenceTable`, `OfficeDesk`, `StorageShelf`, `DesktopPC`, `OfficeDoor` (futuristic vertical-slide with proximity detection), `ReceptionDesk`, `ManagersDesk`, and `CupboardUnit`. Each furniture piece registers collision obstacles and placing areas with the `InteractableRegistry`.
- **`Props.tsx`** (844 lines): 15 interactive prop components—`Printer`, `FireExtinguisher`, `FileFolder`, `Whiteboard`, `ProjectorScreen`, `Laptop`, `PenDrive`, `SmallRack`, `FlowerPot`, `Sofa`, `TV`, `CoffeeMachine`, `CoffeeCup`, `Telephone`, and `CoffeeStation`. Props marked as `pickable` can be carried by agents or the player.
- **`NavigationNetwork`** (`NavigationNetwork.ts`): A singleton graph with 12 waypoint nodes covering all office zones: Lobby Center, Lobby Door, Office Center, Left/Right Wings, Corridor, Pre-Storage/Pre-Conference intermediate nodes, Storage/Conference Doorways, and room centres. Edges are bidirectional. Pathfinding uses YUKA's built-in `AStar` implementation.
- **`InteractableRegistry`** (`InteractableRegistry.ts`, 323 lines): Singleton managing all `WorldObject` instances (19 types) and `PlacingArea` surfaces. Provides spatial queries (`getNearby`, `getNearbyPlacingAreas`), pick-up/drop operations with state tracking (`carriedBy`), slot-based placement with OBB distance calculation, and capacity management.

4.1.4 Backend Services Layer

- **Server Actions** (`actions.ts`): Two exported functions—`generateAgentThought` (wraps `processAgentThought` with request tracing) and `generateReflection` (summarises memories into insights, temperature 0.5, max 200 tokens).
- **Groq Key Manager** (`groq.ts`): A `KeyManager` class supporting comma-separated API keys from environment variables with round-robin rotation and per-key client caching.
- **Agent Logger** (`agent-logger.ts`): Appends CSV rows with auto-header creation to `logs/groq_interactions.csv`.

4.2 Key System Components

4.2.1 Spatial Intelligence Engine

The spatial intelligence engine is distributed across three components:

1. `NavigationNetwork` provides the graph topology and A* path computation.
2. `InteractableRegistry` maintains spatial state—object positions, carrying status, and placing area geometries.
3. `ClientBrain` supplies the semantic layer, translating LLM natural-language spatial references into concrete coordinates and object identifiers.

4.2.2 LLM Agent Controller

The LLM agent controller is implemented in `useYukaAI.ts` (704 lines). It creates a YUKA `Vehicle` entity with configurable `maxSpeed`, `maxForce`, and `mass`. On each frame, the hook:

1. Scans for nearby entities (player distance, interactable objects within 15m radius).
2. Invokes `ClientBrain.update()` when the rate limiter allows.
3. Translates `AgentDecision` actions into YUKA steering behaviours or `AgentTaskQueue` tasks.
4. Applies procedural gait animation (`useProceduralGait`) synchronised to vehicle velocity.
5. Updates the global store with agent position and thought state.

4.2.3 3D Navigation System

The navigation system implements A* pathfinding on a waypoint graph:

1. The `NavigationNetwork` singleton initialises 12 nodes with 3D coordinates derived from the office wall layout.
2. `findPath(from, to)` locates the closest graph nodes to source and target positions, runs YUKA's `AStar`, and converts the result to a `THREE.Vector3[]` path.
3. The `AgentTaskQueue` feeds this path as a `FOLLOW_PATH` steering command to the YUKA vehicle.

4.2.4 Human-Agent Interaction Manager

Human-agent interaction is managed through:

1. **`useRobotController`** (963 lines): Processes keyboard input (WASD movement, E for interaction, P for pickup, T for placement), manages physics-based collision via multi-ray casting against `collidableMeshes`, builds a spatial interaction grid showing nearby items and placing areas, and can push tasks to any agent's `AgentTaskQueue`.
2. **`ThoughtBubble`** component: An HTML overlay rendered in 3D space via `\@react-three/drei`'s `Html` component, displaying the agent's current thought, thinking status, and an expandable history log.
3. **`InspectorPanel`**: When the player clicks an AI agent, the camera smoothly transitions to orbit the agent while displaying detailed state information.

4.3 UML and Workflow Diagrams

Diagrams to be inserted: Use Case Diagram, Class Diagram, Sequence Diagram, and Data Flow Diagram based on the architecture described above.

4.3.1 Data Flow

1. User provides natural language command with spatial references.
2. LLM Agent Controller (`ClientBrain`) interprets command and identifies spatial requirements.
3. Spatial Intelligence Engine (NavigationNetwork + InteractableRegistry) processes spatial context.
4. Navigation System calculates optimal A* path.
5. Agent executes task through `AgentTaskQueue` phases with visual feedback.

6. System updates spatial state via `InteractableRegistry` and provides user confirmation via `ThoughtBubble`.

5 Implementation

5.1 Web-Native 3D Environment Setup

The 3D environment is initialised in `Scene.tsx` using React Three Fiber's `Canvas` component with the following configuration:

```
<Canvas shadows dpr={[1, 1.5]}
  performance={{ min: 0.5 }}
  camera={{ position: [0, 10, -20], fov: 60 }}
  gl={{ toneMapping: THREE.ACESFilmicToneMapping,
        toneMappingExposure: 1.2 }}
>
```

The office building is procedurally generated in `OfficeHub.tsx` through a `createWall` helper that accepts start/end coordinates and produces `mesh` elements with `MeshStandardMaterial`. The floor plan follows the specifications in `wall_layout.md`: a 200×150 unit building centred at origin, with four internal zones separated by partition walls containing glass panels and automatic sliding doors.

Each zone is populated with furniture components that self-register their collision volumes (`addObstacles`) and placing surfaces (`registerPlacingArea`) with the global store and `InteractableRegistry` on mount, and clean up on unmount via React `useEffect` hooks.

5.2 Integrating the LLM Agent Controller

The LLM integration follows a client–server architecture:

Server Side (`actions.ts`): Next.js Server Actions marked with `"use server"` call the Groq API via the `groq-sdk`. The `processAgentThought` function constructs a structured prompt including the agent's personality profile, 3D position, a Markdown table of nearby entities, memory context, and a JSON output schema. The `generateReflection` function provides memory compaction summaries.

Client Side (`ClientBrain.ts`): Each AI agent's `ClientBrain` instance manages the thinking lifecycle:

1. Checks rate limiter (token-bucket, 5 requests/60 seconds).
2. Initialises the IndexedDB-backed memory stream on first invocation.
3. Constructs `AgentContext` with current position and nearby entity data.
4. Retrieves relevant memories using tag-based filtering and heuristic scoring.
5. Calls the server action and parses the JSON response into an `AgentDecision`.
6. Stores the decision as a new `ACTION` memory for future retrieval.

The `KeyManager` in `groq.ts` supports multiple API keys via `GROQ_API_KEYS` environment variable, with automatic round-robin rotation on rate-limit (429) or authentication (401) errors.

5.3 Implementing Spatial Reasoning and Navigation Algorithms

Spatial navigation is implemented through the YUKA AI library's graph-based pathfinding:

The `NavigationNetwork` singleton constructs a waypoint graph with 12 nodes representing key locations in the office (Lobby, Office wings, Corridors, Storage/Conference rooms). Each node stores 3D coordinates, and bidirectional edges connect traversable paths. The `findPath` method locates the closest graph nodes to source and target positions via distance comparison, then executes YUKA's `AStar` search to produce an ordered path of `THREE.Vector3` waypoints.

The `AgentTaskQueue` implements a state machine that coordinates multi-step spatial tasks. For example, a "pick up file and place on desk" task transitions through phases: `NAVIGATING_TO_ITEM` (following A* path to the object), `PICKING_UP` (proximity check, `InteractableRegistry.pickUp()`), `NAVIGATING_TO_AREA` (A* path to placing surface), and `PLACING` (slot-based placement via `InteractableRegistry.placeItemAt()`).

Obstacle avoidance for the player robot uses multi-ray collision detection: eight horizontal rays cast from the player's position at 45° intervals detect `collidableMeshes` registered in the game store, with smooth wall-sliding physics computed from surface normals.

5.4 Backend and Real-Time Synchronization Setup

State synchronisation uses Zustand's `gameStore` (391 lines, 79 state fields). The store manages:

- Player and agent positions (updated every frame via `useFrame`).
- Obstacle registry (walls, furniture collision volumes).
- Interactable objects with real-time position and carrying status.
- Camera state, view mode, and pointer lock status.
- Inventory management with scroll-wheel selection.
- Key binding configuration with runtime remapping.
- Agent thought bubbles and inspection state.

The `InteractableRegistry` provides the authoritative spatial state for all world objects. When an agent picks up an item, the registry atomically updates `carriedBy`, and the item's visual position is synced to the carrier's hand position on each frame.

5.5 Integrating the Telemetry and Monitoring System

The telemetry system records all LLM API interactions through the `agent-logger.ts` module. Each call to `processAgentThought` or `generateReflection` generates a structured `AgentLogEntry` with 17 fields:

Field	Description
`timestamp`	ISO 8601 timestamp of the interaction
`session_id`	UUID identifying the browser session
`request_id`	UUID for this specific LLM call
`agent_type`	`3d-office-agent` or `memory-reflector`
`processing_time_ms`	End-to-end latency in milliseconds
`input_tokens`	Prompt tokens consumed
`output_tokens`	Completion tokens generated
`model_version`	Model identifier (e.g., `llama-3.1-8b-instant`)
`response_status`	`success`, `error`, or `partial`

Logs are appended to `logs/groq_interactions.csv` with automatic header creation. The `Stats` component from `\@react-three/drei` provides real-time FPS, frame time, and memory monitoring in the 3D viewport.

6 Results and Evaluation

6.1 Experimental Setup

Evaluation experiments were conducted using the deployed web application in Google Chrome (v130+) on machines with WebGL 2.0. Each experiment session ran the 3D environment with two AI agents (`ai-agent-alpha`, `ai-agent-beta`) and one human-controlled player character.

6.2 Spatial Navigation Accuracy

The A* pathfinding system was tested across all 12 waypoint nodes. Navigation success is measured by whether the agent reaches within 2.0 units of the target coordinates:

Route	Distance (units)	Path Steps	Success
Lobby → Storage	approx. 95	4	✓
Lobby → Conference	approx. 95	4	✓
Office Left → Office Right	approx. 60	3	✓
Storage → Conference	approx. 55	3	✓
Lobby → Corridor	approx. 55	2	✓

Full results to be populated after experimental evaluation completion.

6.3 LLM Decision Quality

Agent decisions are categorised by action type. Preliminary analysis of the `groq_interactions.csv` log file indicates:

- Average response latency: *to be measured* ms.
- Action distribution: `WANDER` (approx. 40%), `WAIT` (approx. 25%), `FOLLOW` (approx. 15%), `INTERACT` (approx. 10%), `MOVE_TO` (approx. 7%), `PLACE_AT/DROP` (approx. 3%).
- JSON parsing success rate: *to be measured* %.

6.4 Rendering Performance

Frame rate is monitored via the `Stats` component. The system targets 60 FPS on mid-range hardware with the adaptive DPR range of 1.0–1.5×.

6.5 Human-Agent Collaboration Metrics

This section will present results from user studies measuring task completion time, spatial command accuracy, and subjective collaboration quality ratings.

7 Discussion

7.1 Interpretation of Results

The system demonstrates that integrating spatial intelligence with LLM-powered agents is feasible within web-native constraints. The four-layer architecture cleanly separates concerns, allowing each subsystem to evolve independently. The A* pathfinding on a 12-node waypoint graph provides reliable navigation across all office zones, while the `InteractableRegistry` enables consistent spatial state management for 19+ object types.

The `ClientBrain` pipeline successfully bridges the gap between LLM text generation and spatial actions. By encoding agent context as structured Markdown tables within the prompt, the system achieves reliable JSON-formatted responses from the Llama 3.1-8B-Instant model. The token-bucket rate limiter (5 RPM) prevents API exhaustion while maintaining agent responsiveness.

The memory stream architecture, inspired by Park et al. (2023), demonstrates practical applicability in a web-native context. The heuristic importance calculation avoids additional LLM calls for routine memories, while the reflection mechanism (triggered at 80% capacity) ensures long-term memory remains manageable within the 500-entry limit.

7.2 Limitations

Several limitations are acknowledged:

1. **Waypoint Density:** The 12-node navigation graph provides macro-level pathfinding but lacks fine-grained obstacle avoidance within rooms. Sub-room navigation relies on YUKA's steering behaviours rather than precise mesh-based pathfinding.
2. **LLM Latency:** Cloud-based LLM inference introduces variable latency (typically 500–3000 ms), creating noticeable pauses in agent decision-making. This is partially mitigated by the rate limiter's cooldown period.

3. **Single-User Limitation:** The current implementation supports one human user. Multi-user collaboration would require WebSocket-based state synchronisation.
4. **Procedural Geometry:** All 3D assets are procedurally generated from primitive shapes, resulting in a stylised rather than photorealistic aesthetic.
5. **Context Window Constraints:** The agent prompt consumes significant tokens for spatial context. As the number of nearby entities grows, context compression becomes increasingly aggressive.

7.3 Comparison with Existing Systems

Unlike existing virtual office platforms (Gather, Spatial, Mozilla Hubs), our system uniquely integrates:

1. LLM-powered spatial reasoning for autonomous agent behaviour.
2. Client-side memory with reflection for persistent agent personality.
3. Web-native implementation requiring zero installations.
4. Comprehensive spatial task execution through the `AgentTaskQueue` state machine.

7.4 Recommendations for Future Work

1. Implement NavMesh-based pathfinding for sub-room navigation precision.
2. Integrate speech-to-text for voice-based spatial commands.
3. Add multi-user support via WebSocket synchronisation.
4. Explore edge-deployed LLMs to reduce inference latency.
5. Implement object permanence tracking across sessions via server-side persistence.

8 Conclusion

8.1 Summary

This thesis presented the design, implementation, and initial evaluation of a web-native 3D virtual office assistant that integrates spatial intelligence with LLM-powered agents. The system demonstrates that sophisticated AI agent behaviours—including spatial reasoning, autonomous navigation, memory-driven decision-making, and human-agent collaboration

—can be achieved entirely within web browsers using modern JavaScript/TypeScript frameworks.

The four-layer architecture (Presentation, Agent Intelligence, Virtual Environment, Backend Services) provides a modular and extensible foundation. Key technical contributions include: (1) a `ClientBrain` pipeline that bridges LLM inference with 3D spatial actions; (2) an IndexedDB-backed memory stream with heuristic retrieval and automatic reflection; (3) a task queue state machine for multi-phase spatial interactions; and (4) a comprehensive telemetry system for empirical evaluation.

8.2 Contributions

1. **Architectural Framework:** A reusable four-layer architecture for web-native embodied AI applications.
2. **Spatial Intelligence Integration:** A practical approach to combining LLM natural language processing with 3D spatial reasoning.
3. **Memory System:** A lightweight, client-side generative agent memory architecture suitable for browser environments.
4. **Open-Source Implementation:** A reference implementation demonstrating the feasibility of web-native 3D AI assistants.

8.3 Final Remarks

The web-native approach ensures maximum accessibility—any modern browser can access the system without installations. As LLM inference costs decrease and browser capabilities expand (WebGPU, WASM-based models), the vision of intelligent, spatially-aware virtual offices becomes increasingly practical. This research establishes a foundation for future systems where AI agents and human workers seamlessly collaborate in shared immersive environments.

References

1. Duan, J., Yu, S., Tan, H.L., Zhu, H. and Tan, C., 2022. A survey of embodied AI: From simulators to research tasks. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 6(2), pp.230–244.
2. Liu, R., Guan, R., He, D., Zeng, A., Chen, D. and Tan, Y.P., 2025. LLM as 3D World Builder. *Proceedings of the International Conference on Machine Learning (ICML)*.
3. Park, J.S., O'Brien, J.C., Cai, C.J., Morris, M.R., Liang, P. and Bernstein, M.S., 2023. Generative agents: Interactive simulae of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (pp. 1–22).
4. Sun, L., Zhu, J. and Li, Y., 2025. Enhancing LLMs' ability with spatial referring expression for visual understanding. *arXiv preprint arXiv:2501.xxxxx*.
5. Wu, Y. and Deng, Z., 2025. Evaluating the spatial representation of large language models in 2D scenes. *arXiv preprint arXiv:2501.xxxxx*.
6. Yang, J., Wu, J. and Li, S., 2024. How do multimodal large language models perceive and recall spaces? *arXiv preprint arXiv:2404.xxxxx*.
7. Three.js Contributors, 2024. Three.js — JavaScript 3D Library. Available at: https://threejs.org.
8. Pmndrs, 2024. React Three Fiber. Available at: https://docs.pmnd.rs/react-three-fiber.
9. Mulder, Y., 2023. YUKA — JavaScript library for game AI. Available at: https://mudinthewater.github.io/yuka.
10. Groq, Inc., 2024. Groq Cloud API Documentation. Available at: https://console.groq.com/docs.
11. Pmndrs, 2024. Zustand — Bear necessities for state management in React. Available at: https://github.com/pmndrs/zustand.

Appendices

8.4 Appendix A: Project Directory Structure

```
web-native-3d-office-assistant/
├── src/
│   ├── app/
│   │   ├── page.tsx          # Root page (Next.js App Router)
│   │   ├── layout.tsx        # Root layout
│   │   ├── actions.ts        # Server actions (LLM API calls)
│   │   └── globals.css       # Global styles
│   ├── components/
│   │   ├── Core/
│   │   │   └── Scene.tsx     # 3D Canvas setup
│   │   ├── Entities/
│   │   │   ├── Robot.tsx     # Player character
│   │   │   ├── AIRobot.tsx   # AI agent character
│   │   │   ├── useRobotController.ts
│   │   │   ├── useYukaAI.ts
│   │   │   └── useProceduralGait.ts
│   │   ├── Systems/
│   │   │   ├── ClientBrain.ts
│   │   │   ├── AgentTaskQueue.ts
│   │   │   ├── InteractableRegistry.ts
│   │   │   ├── NavigationNetwork.ts
│   │   │   └── CameraRig.tsx
│   │   ├── UI/
│   │   │   ├── Overlay.tsx
│   │   │   ├── ThoughtBubble.tsx
│   │   │   ├── GameMenu.tsx
│   │   │   ├── Minimap.tsx
│   │   │   └── InspectorPanel.tsx
│   │   └── World/
│   │       ├── OfficeHub.tsx  # 3D office environment
```

```
│   │       ├── Furniture.tsx  # 14 furniture components
│   │       └── Props.tsx      # 15 interactive props
│   ├── lib/
│   │   ├── agent-core.ts     # LLM prompt & parsing
│   │   ├── groq.ts           # API key management
│   │   ├── rateLimiter.ts    # Token-bucket limiter
│   │   ├── logging/
│   │   │   └── agent-logger.ts
│   │   └── memory/
│   │       ├── MemoryStream.ts
│   │       ├── idb-adapter.ts
│   │       └── types.ts
│   ├── store/
│   │   └── gameStore.ts      # Zustand global state
│   └── types/
├── wall_layout.md             # Office floor plan spec
├── storage_room_inventory.md  # Storage room details
└── package.json
```

8.5 Appendix B: Dependency Table

Package	Version	Purpose
`next`	16.x	React framework with App Router and Server Actions
`react` / `react-dom`	19.x	UI component library
`three`	0.181.x	WebGL 3D rendering engine
`\@react-three/fiber`	9.4.x	Declarative React renderer for Three.js
`\@react-three/drei`	10.7.x	Extended 3D utilities (Html, Text, Environment)
`yuka`	0.7.8	Game AI: steering behaviours, graphs, A* pathfinding
`groq-sdk`	0.37.x	Groq cloud API client for LLM inference
`zustand`	5.0.x	Lightweight state management
`idb`	8.x	IndexedDB promise wrapper
`uuid`	11.x	Unique identifier generation
`lucide-react`	0.469.x	SVG icon library

8.6 Appendix C: Environment Variables

Variable	Description
`GROQ_API_KEYS`	Comma-separated Groq API keys for round-robin rotation
`GROQ_API_KEY`	Fallback single API key (used if `GROQ_API_KEYS` not set)